To verify the effectiveness of our AutoPT architecture on the end-to-end penetration testing task, we conduct independent validation experiments on the test data sets we collected. Specifically, we independently tested each vulnerability environment five times, recorded the results and necessary logs, and initialized the entire system for the next experiment. The experimental results are shown in Table 4. In general, the existing large language models have sufficient capabilities to complete most simple end-to-end penetration testing tasks. However, they still perform average on tasks with more operation steps. Although GPT-4o mini demonstrates a higher overall success rate, completing 40% of the total tasks, it completes only 20% of the complex tasks. In contrast, the more advanced GPT-4o model completes 40% of these complex tasks.

During the experiment, we found that in the Agent state, each agent solves a relatively simple subtask, which has a higher success rate than directly solving complex end-to-end tasks. Notably, the Rule state, as expected, successfully assisted the Agent state in focusing on the core vulnerability information, enabling it to perform well in both the query and vulnerability exploitation subtasks.

Answering RQ1: AutoPT effectively completes most end-to-end penetration testing tasks. The results show that even if the model capability is slightly weaker, the AutoPT architecture has strong automated penetration testing capabilities.

6.3 Performance Evaluation (RQ2)

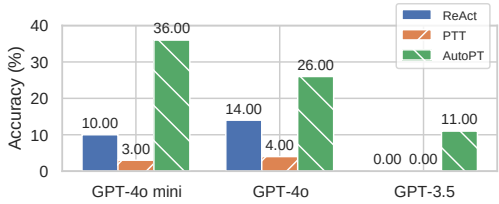

Fig. 5. Overall performance of agents based on the GPT-3.5, GPT-4o, and GPT-4o mini models in the ReAct, PTT, and AutoPT architectures.

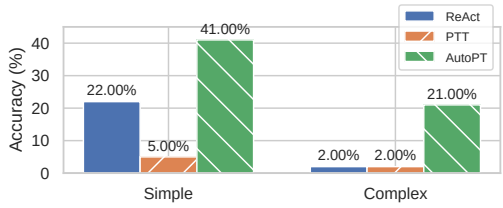

Fig. 6. Comparison of average performance of agents based on the GPT-4o and GPT-4o mini models on tasks of varying complexity on the ReAct, PTT, and AutoPT architectures.

We compare the overall end-to-end penetration testing task completion of AutoPT-GPT-3.5, AutoPT-GPT-4o, and AutoPT-GPT-4o-mini with the performance of the three models under the two frameworks, ReAct and PTT constructed. As shown in Figure 5, compared with the framework built in the previous section, our solution supported by LLM demonstrates extraordinary vulnerability testing capabilities. Specifically, AutoPT-GPT-4o-mini far outperforms the other solutions and even under our method. It is worth noting that even the worst GPT-3.5 model has completed 11% experimental samples, achieves a leap from 0 to 1, and even completes more tasks than other architectures of the GPT-4o and GPT-4o mini models. This performance shows that our solution can compensate for some of the model's capacity deficiencies through the advantages of the architecture, and it can be seen that our approach solves **Challenge 3**.

The results supported by the GPT-4o mini even achieved a success rate of 36%, which shows that our solution has a very high upper limit in end-to-end penetration testing. We calculated the average performance of AutoPT-GPT-4o and AutoPT-GPT-4o-mini on tasks of different complexity. Then we compared them with the average performance of GPT-4o and GPT-4o mini under the two frameworks described in detail in Section 4.2. As shown in Figure 6, our solution performs better than the other two solutions on all tasks. It is worth noting that compared with ReAct, AutoPT not only doubled the number of completions on simple tasks but also achieved nearly

10 times the number of completions on difficult tasks. This highlights that our design effectively alleviates the problems encountered by the current end-to-end tasks mentioned above, bringing more promising test results. Compared with the ReAct framework, our method splits the task into subtasks, allowing the model to focus on simple and clear tasks. This also reduces the generation of erroneous commands and alleviates hallucination problems, maximizing the model's capabilities.

Answering RQ2: In the end-to-end penetration testing task, the success rate of the AutoPT architecture is significantly better than that of other agent frameworks. The success rate doubles for simple tasks and nearly 10 times for complex tasks.

6.4 Cost Evaluation (RQ3)

Table 5. Comparison of money and time cost.

Arch	AutoPT	ReAct	PTT	Human
Money	$0.99325	$3.49266	$4.12331	$310
Time	4.48h (16,131.07s)	8.81h (31,730.98s)	10.83h (38,997.49s)	about 5h -

Based on the results in the previous section, as shown in Table 5, we now analyze the cost of performing end-to-end penetration tests with GPT-4o mini (with the highest success rate) and compare it to a separate manual penetration test. These analyses are not intended to show the exact cost of a real hacker attack on a website but rather to highlight the economic feasibility of building AutoPT and using AutoPT to perform end-to-end penetration testing. To estimate the cost of AutoPT, we calculate the average duration and average API cost of all the experimental architectures driven by GPT-4o mini. In all 20 experiments, the total cost is $0.99325, the average cost is $0.00993, the total time is 16131.07 seconds, and the average time is 161.31 seconds. The overall success rate was 41.00%, totaling $0.02423 per website.

Our AutoPT significantly reduces the money and time costs. Here we emphasize several features of end-to-end LLM. First, AutoPT increases the success rate of tasks and reduces redundant operations through state machine jumps. In the future, costs can be further reduced through a more optimized Agent architecture. Second, LLM-driven agents can work without restrictions on time and location. Third, since the creation of large language models, the cost of API requests for large language models has continued to decline. Finally, the capabilities of open-source models are also constantly improving. In the future, the deployment of local models can further reduce the time cost caused by network delays.

We further compared AutoPT to the costs of human penetration workers. A detailed analysis of the costs of manual penetration requires an understanding of the specific internal structure of hacker organizations, which is beyond the scope of this article. Unlike other tasks (such as classification tasks), penetration testing requires professional knowledge and cannot be completed by non-experts. We first estimated the penetration testing operation time for this task. When building the selection task in Section 3, we manually reproduced all 20 vulnerabilities, and it took an average of 5 man-hours to complete all vulnerability reproductions. On the basis of the average salary of network penetration testers in 2024 of $124,000 [4], the cost is estimated to be approximately $62 per hour based on a standard working time of 40 hours per week and 50 weeks per year, and the total cost is approximately $310. This cost is approximately 300 times greater than that of AutoPT.

We emphasize that these calculations are intended to provide an estimate of the overall cost, and the results of the comparison are rough approximations. Nevertheless, our analysis reveals a large cost difference between human experts and LLM-based agents. We expect these costs to be further reduced with the development of LLM. In addition, future research may require the development of more efficient and targeted agent frameworks to cope with highly specialized end-to-end penetration testing tasks.

[4] https://isecjobs.com/salaries/penetration-tester-salary-in-2024

Answering RQ3: Experimental results show that AutoPT reduces time by 10% and economic cost by 99.6% compared with humans and reduces time by 50% and economic cost by 71.6% compared with other LLM-based frameworks.

7 VALIDITY ANALYSIS

7.1 Internal Threats

The first potential threat to internal validity involves the performance of the AutoPT architecture. To mitigate this issue, we thoroughly verified the source code used in the original method to minimize errors.

Second, internal validity involves the accuracy of the scanner. We utilized Xray, an open-source scanner, and used all the scanning POCs. However, potential configuration errors or improper configurations may lead to inaccurate or incomplete scanning results. To address this threat, we manually configured and carefully checked all Xray scan results to minimize errors.

In addition, our method did not make further attempts in detail. For example, although we used jailbreaking methods [66] to bypass model alignment, we did not try more powerful and hidden jailbreaking methods. Similarly, since the advent of large models, hundreds or thousands of articles have been published on the large model hallucination problem. Although our method has a certain effect on the hallucination problem from the aspect of agent architecture, it does not make an in-depth attempt to solve the agent hallucination problem.

7.2 External Threats

The initial external threat to effectiveness stems from the limitation of being able to configure only the vulnerability environment, which may impact the entire end-to-end penetration testing task. However, we use a docker reproduction environment from Vulhub, one of the most authoritative vulnerability reproduction platforms. Moreover, we manually tested its availability and vulnerability item by item, which greatly mitigated the threat.

The second external threat to validity is that the reference link information queried by the model may be outdated or erroneous, thereby misleading the model in solving the task. Our mitigation method involves manually screening the reference link content to ensure that the queried information is key information related to vulnerability exploitation.

8 DISCUSSION AND LIMITATION

Discussion. Since the advent of ChatGPT, the use of large language model capabilities in network security has attracted the attention of researchers. Many black-hat and white-hat practitioners are also trying to use the capabilities of large language models in their work. Therefore, we expect that automated network attacks driven by LLMs will increase and that the speed and efficiency of these attacks will be significantly accelerated.

Although AutoPT performed well in terms of the experimental results, we must emphasize that, based on the current model capabilities, we are still some distance away from a fully automated penetration testing system in the real world. On the other hand, the large language model security team sets network security issues as violations to prevent hacker crimes, artificially increasing the difficulty of using large language models for security attacks and defense research.

Limitation and future work. 1) The purpose of this study is to evaluate the feasibility of using LLM-based agents to automatically perform end-to-end penetration testing. As in previous work, the victim environment has been configured to be insecure before the attack (e.g., default dangerous configuration). In addition, in this work, we focus on the end-to-end ability of LLMs to exploit vulnerabilities, and we did not attempt the more important vulnerability mining direction. 2) As

mentioned in the previous section, enabling the agent to perform simulated web page operations, which some companies and researchers have begun to attempt [59], is an important factor in mitigating specific operations. 3) The ideas proposed in this paper may be used by real-world attackers. In the future, we need to consider defense work against AutoPT, such as identifying whether the request command is an LLM-driven network attack through LLM hallucination detection [11, 37].

9 CONCLUSION

In this work, we first define **the end-to-end penetration testing** task. Then, we conduct pre-experiments, select models, and comprehensively try to summarize the capabilities and limitations of common agent architectures in the context of end-to-end penetration testing tasks. We find that agents are able to solve basic penetration testing tasks and are able to exploit testing tools successfully. Moreover, they also face challenges such as difficulty maintaining historical messages and agents stuck.

Based on these findings, we designed a novel agent architecture of **PSM** inspired by **FSM**. Then, we adopted a divide-and-conquer approach and built the **AutoPT** system using PSM. To the best of our knowledge, this is the first LLM-based attempt for end-to-end penetration testing tasks. Our comprehensive evaluation of AutoPT demonstrates its potential and value in academia and industry. Ultimately, our paper aims to draw attention and stimulate thinking about a pressing research question: *How far are we from end-to-end automated web penetration testing?* Overall, the contributions of this research are valuable resources and provide a promising direction for continued research and development in advanced automation for penetration testing.

DATA AVAILABILITY

To promote open science, we provided a Github [5] for future work. This includes all benchmark data, as well as the code used for pre-experiments and implementation of AutoPT.

REFERENCES

[1] 2023. HackTheBox. https://www.hackthebox.com.
[2] Farah Abu-Dabaseh and Esraa Alshammari. 2018. Automated penetration testing: An overview. In *The 4th international conference on natural language computing, Copenhagen, Denmark*. 121–129.
[3] Meta AI. 2024. Meta AI Blog: Meta LLaMA 3.1. https://ai.meta.com/blog/meta-llama-3-1/.
[4] Anthropic. 2024. *Introducing Claude 3.5 Sonnet*. https://www.anthropic.com/news/claude-3-5-sonnet
[5] Dennis Appelt, Cu Duy Nguyen, Lionel C Briand, and Nadia Alshahwan. 2014. Automated testing for SQL injection vulnerabilities: an input mutation approach. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. 259–269.
[6] Brad Arkin, Scott Stender, and Gary McGraw. 2005. Software penetration testing. *IEEE Security & Privacy* 3, 1 (2005), 84–87.
[7] Nor Fatimah Awang and Azizah Abd Manaf. 2013. Detecting vulnerabilities in web applications using automated black box and manual penetration testing. In *International Conference on Security of Information and Communication Networks*. Springer, 230–239.
[8] Kevin Bock, George Hughey, and Dave Levin. 2018. King of the Hill: A Novel Cybersecurity Competition for Teaching Penetration Testing. In *2018 USENIX Workshop on Advances in Security Education (ASE 18)*. USENIX Association, Baltimore, MD. https://www.usenix.org/conference/ase18/presentation/bock
[9] Tanner J Burns, Samuel C Rios, Thomas K Jordan, Qijun Gu, and Trevor Underwood. 2017. Analysis and exercises for engaging beginners in online {CTF} competitions for security education. In *2017 USENIX Workshop on Advances in Security Education (ASE 17)*.
[10] Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain Version 0.2.34, Accessed: 2024-08-21.
[11] Yuyan Chen, Qiang Fu, Yichen Yuan, Zhihao Wen, Ge Fan, Dayiheng Liu, Dongmei Zhang, Zhixu Li, and Yanghua Xiao. 2023. Hallucination detection: Robustly discerning reliable answers in large language models. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 245–255.

[5] https://github.com/Dizzy-K/AutoPT

[12] PCI Security Standards Council. 2017. Information Supplement: Penetration Testing Guidance. https://www.pcisecuritystandards.org/documents/Penetration-Testing-Guidance-v1_1.pdf Accessed: 2023-08-24.

[13] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2023. PentestGPT: An LLM-empowered Automatic Penetration Testing Tool. arXiv:2308.06782 [cs.SE]

[14] Gelei Deng, Zhiyi Zhang, Yuekang Li, Yi Liu, Tianwei Zhang, Yang Liu, Guo Yu, and Dongjin Wang. 2023. {NAUTILUS}: Automated {RESTful}{API} Vulnerability Detection. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5593–5609.

[15] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.

[16] Adam Doupé, Ludovico Cavedon, Christopher Kruegel, and Giovanni Vigna. 2012. Enemy of the State: A State-Aware Black-Box Web Vulnerability Scanner. In *21st USENIX Security Symposium (USENIX Security 12)*. USENIX Association, Bellevue, WA, 523–538. https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/doupe

[17] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[18] OpenAI et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[19] Marius Fleischer, Dipanjan Das, Priyanka Bose, Weiheng Bai, Kangjie Lu, Mathias Payer, Christopher Kruegel, and Giovanni Vigna. 2023. {ACTOR}:{Action-Guided} Kernel Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5003–5020.

[20] Georgios Giantamidis, Stavros Tripakis, and Stylianos Basagiannis. 2021. Learning Moore machines from input–output traces. *International Journal on Software Tools for Technology Transfer* 23, 1 (2021), 1–29.

[21] Hao Guan, Guangdong Bai, and Yepang Liu. 2024. Large Language Models Can Connect the Dots: Exploring Model Optimization Bugs with Domain Knowledge-Aware Prompts. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1579–1591.

[22] Emre Güler, Sergej Schumilo, Moritz Schloegel, Nils Bars, Philipp Görz, Xinyi Xu, Cemal Kaygusuz, and Thorsten Holz. 2024. Atropos: Effective fuzzing of web applications for server-side vulnerabilities. In *USENIX Security Symposium*.

[23] William GJ Halfond, Saswat Anand, and Alessandro Orso. 2009. Precise interface identification to improve testing and analysis of web applications. In *Proceedings of the eighteenth international symposium on Software testing and analysis*. 285–296.

[24] Andreas Happe and Jürgen Cito. 2023. Getting pwn'd by AI: Penetration Testing with Large Language Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM. https://doi.org/10.1145/3611643.3613083

[25] Andreas Happe and Jürgen Cito. 2023. Understanding Hackers' Work: An Empirical Study of Offensive Security Practitioners. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, 1669–1680. https://doi.org/10.1145/3611643.3613900

[26] Andreas Happe and Jürgen Cito. 2023. Understanding Hackers' Work: An Empirical Study of Offensive Security Practitioners. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1669–1680.

[27] Andreas Happe, Aaron Kaplan, and Juergen Cito. 2024. LLMs as Hackers: Autonomous Linux Privilege Escalation Attacks. arXiv:2310.11409 [cs.CR] https://arxiv.org/abs/2310.11409

[28] Marzuki Hasibuan and Andi Marwan Elhanafi. 2022. Penetration Testing Sistem Jaringan Komputer Menggunakan Kali Linux untuk Mengetahui Kerentanan Keamanan Server dengan Metode Black Box: Studi Kasus Web Server Diva Karaoke. co. id. *SUDO Jurnal Teknik Informatika* 1, 4 (2022), 171–177.

[29] Zhenguo Hu, Razvan Beuran, and Yasuo Tan. 2020. Automated penetration testing using deep reinforcement learning. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2–10.

[30] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232* (2023).

[31] Sadeeq Jan, Cu D Nguyen, and Lionel C Briand. 2016. Automated and effective testing of web services for XML injection attacks. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*. 12–23.

[32] Haibo Jin, Ruoxi Chen, Andy Zhou, Jinyin Chen, Yang Zhang, and Haohan Wang. 2024. GUARD: Role-playing to generate natural-language jailbreakings to test guideline adherence of large language models. *arXiv preprint arXiv:2402.03299* (2024).

[33] Nickolaos Koroniotis, Nour Moustafa, Benjamin Turnbull, Francesco Schiliro, Praveen Gauravaram, and Helge Janicke. 2021. A deep learning-based penetration testing framework for vulnerability identification in internet of things environments. In *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications*